LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 7 – Abstract & Interface



NAMA: ANDITA AYUNINGTYAS

NIM: 103132400016

KELAS: PS1DS-05-REG-01

PROGRAM STUDI S1 SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Abstract class merupakan class dalam Java yang dideklarasikan menggunakan keyword abstract dan berfungsi sebagai kelas dasar bagi class turunannya. Abstract class tidak dapat dibuat menjadi objek secara langsung, tetapi dapat diwariskan menggunakan keyword extends. Class ini dapat memiliki atribut, constructor, method biasa, dan abstract method. Abstract method merupakan method yang hanya memiliki deklarasi tanpa implementasi sehingga harus didefinisikan kembali (overriding) oleh class turunannya.

Interface merupakan struktur dalam Java yang digunakan untuk mendefinisikan kumpulan method yang harus diimplementasikan oleh class menggunakan keyword implements. Method dalam interface secara default bersifat public abstract, sedangkan variabelnya bersifat public static final. Interface juga mendukung penggunaan default method yang memiliki implementasi dan static method yang dapat dipanggil secara langsung melalui nama interface.

Abstract class dan interface memiliki fungsi yang berbeda, tetapi dapat digunakan secara bersamaan dalam sebuah program. Abstract class berfungsi menyediakan atribut dan perilaku umum bagi class turunannya, sedangkan interface berfungsi menetapkan aturan method yang harus diterapkan oleh suatu class. Kedua konsep ini membantu membangun program Java yang lebih terstruktur serta mendukung penerapan inheritance, polymorphism, dan method overriding.

B. Guided

1. Guided 1

a. [Employee]

```
/*

 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license

 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template

 */

package andita.projectmodul07.guided1;


/**

 *
```

```
 * @author andit
 */
public abstract class Employee {
    private String name;

    public Employee(String name){
        this.name = name;
    }

    public String getName(){
        return name;
    }

    //setiap pegawai pasti punya gaji, tapi rumusnya beda-beda
    public abstract double earnings();

}
```

b. [SalariedEmployee]

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package andita.projectmodul07.guided1;

/**
 *
 * @author andit
 */

//wajib menginput method yang sudah dideklarasikan
public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary){
```

```
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings(){
        return weeklySalary;
    }
}
```

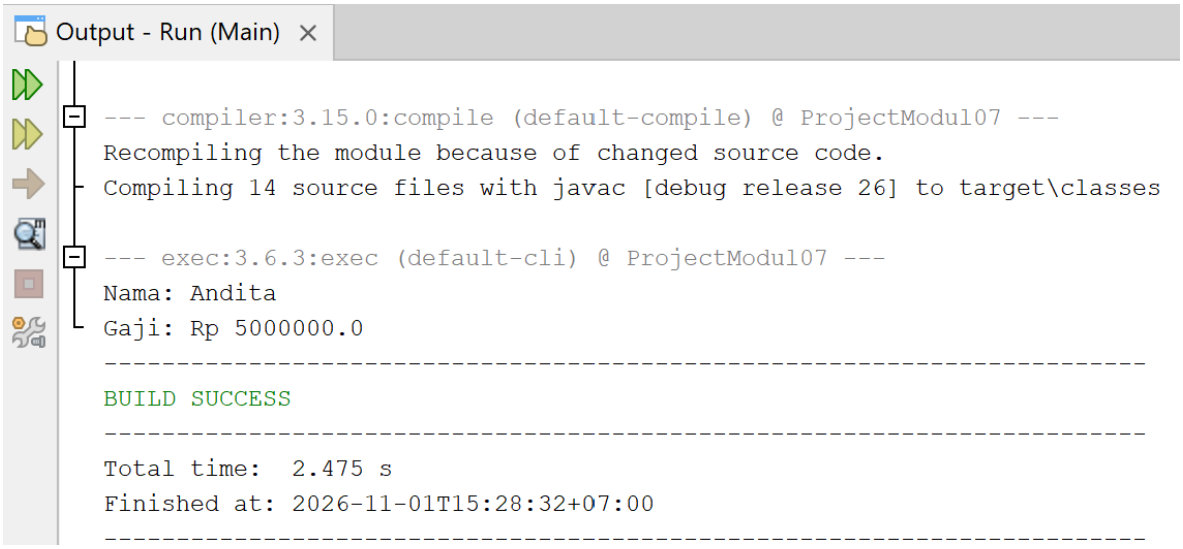
c. [Main]

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package andita.projectmodul07.guided1;

/**
 *
 * @author andit
 */
public class Main {

    public static void main(String[]args){
        SalariedEmployee salaried = new SalariedEmployee("Andita", 5000000);
        System.out.println("Nama: "+salaried.getName());
        System.out.println("Gaji: Rp "+ salaried.earnings());
    }
}
```

Output :



2. Guided 2

a. [Employee]

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package andita.projectmodul07.guided2;

/**
 *
 * @author andit
 */

//fondasi umum yang belum memiliki peran spesifik sehingga tidak boleh diinstansiasi langsung
public abstract class Employee implements Payable{

    private String name;

    //constructur
    public Employee(String name) {
        this.name = name;
    }

    //getter
    public String getName() {
        return name;
    }
```

```
    //method, Setiap pegawai pasti punya gaji, tapi rumusnya berbeda-beda
    public abstract double earnings();

    //saat ada penagihan, alihkan langsung ke metode kalkulasi gaji (earnings)
    @Override
    public double getPaymentAmount(){
        return earnings();
    }
}
```

b. [Invoice]

```
    /*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
 */
package andita.projectmodul07.guided2;

/**
 *
 * @author andit
 */

//Benda mati non-pegawai, tapii tetap diikat oleh aturan
public class Invoice implements Payable {
    private  String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double pricePerItem){
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName(){
        return partName;
    }

    @Override
    public double getPaymentAmount(){
        return quantity * pricePerItem;
    }
}
```

c. [Payable]

```
/*
```

```
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
 */
package andita.projectmodul07.guided2;

/**
 *
 * @author andit
 */

//standar sistem: siapapu
public interface Payable {
    double getPaymentAmount();

}
```

d. [SalariedEmployee]

```
    /*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
 */
package andita.projectmodul07.guided2;

/**
 *
 * @author andit
 */

//wajib menginput method yang sudah dideklarasikan
public class SalariedEmployee extends Employee {

    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }
}
```

e. [Main]

```
   /*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
 */
package andita.projectmodul07.guided2;

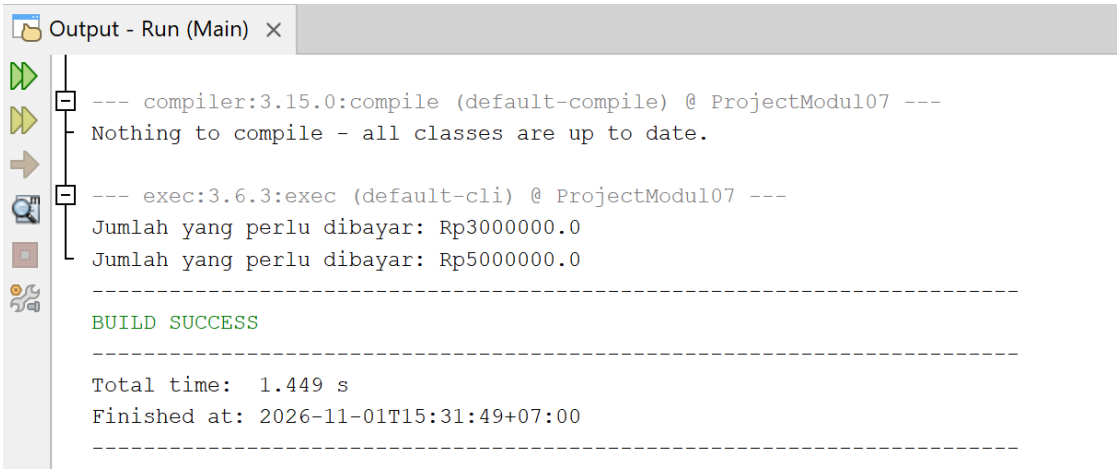
/**
 *
 * @author andit
 */

public class Main {
   public static void main(String[]args){
      Payable[] items = new Payable[2];
      items[0] = new Invoice("Monitor LED", 2, 1500000);
      items[1] = new SalariedEmployee("Andita", 5000000);

      //Dynamic dispatch: Java otomatis mengeksekusi rumus yang tepat
      for (Payable item : items){
         System.out.println("Jumlah yang perlu dibayar: Rp" + item.getPaymentAmount());

      }
   }
}
```

Output :



3. Guided 3

a. [Employee]

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package andita.projectmodul07.guided3;

/**
 *
 * @author andit
 */
//fondasi umum yang belum memiliki peran spesifik sehingga tidak boleh diinstansiasi langsung
public abstract class Employee implements Payable{

    private String name;

    //constructur
    public Employee(String name) {
        this.name = name;
    }

    //getter
    public String getName() {
        return name;
    }

    //method, Setiap pegawai pasti punya gaji, tapi rumusnya berbeda-beda
    public abstract double earnings();

    //saat ada penagihan, alihkan langsung ke metode kalkulasi gaji (earnings)
    @Override
    public double getPaymentAmount(){
        return earnings();
    }
}
```

b. [Invoice]

```
/*
```

```
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package andita.projectmodul07.guided3;

/**
 *
 * @author andit
 */

//Benda mati non-pegawai, tapii tetap diikat oleh aturan
public class Invoice implements Payable {
    private  String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double pricePerItem){
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName(){
        return partName;
    }

    @Override
    public double getPaymentAmount(){
        return quantity * pricePerItem;
    }
}
```

c. [Payable]

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package andita.projectmodul07.guided3;

/**
```

```
 *
 * @author andit
 */
public interface Payable {
    double getPaymentAmount();

    // Fitur baru
    default void printReceipt(){
        System.out.println("Struk Resmi - Total: Rp" + getPaymentAmount());
    }

    // Fungsi utilitas
    static void logTransaction(String message){
        System.out.println("[AUDIT LOG] " + message);
    }
}
```

d. [SalariedEmployee]

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package andita.projectmodul07.guided3;

/**
 *
 * @author andit
 */
//wajibb menginput method yang sudah dideklarasikan
public class SalariedEmployee extends Employee {

    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }
```

```
}
```

e. [Main]

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package andita.projectmodul07.guided3;

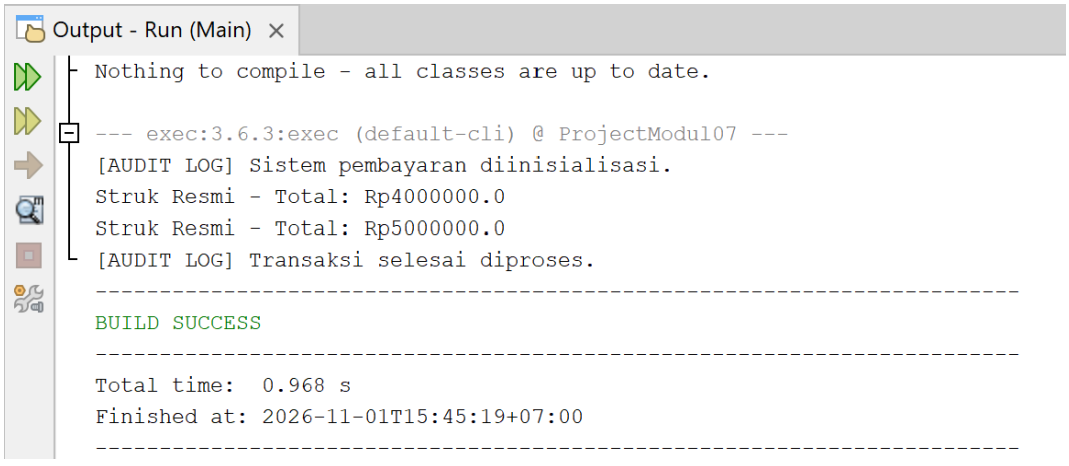
/**
 *
 * @author andit
 */
public class Main {
    public static void main(String[] args) {
        Payable.logTransaction("Sistem pembayaran diinisialisasi.");

        Payable[] items = new Payable[2];
        items[0] = new Invoice("Ram DDR 5 16GB", 2, 2000000);
        items[1] = new SalariedEmployee("Andita", 5000000);

        for (Payable item : items){
            item.printReceipt();
        }

        Payable.logTransaction("Transaksi selesai diproses.");
    }
}
```

Output :



C. Unguided

1. Soal Unguided1

mahasiswa diminta membuat program Cinema POS (Point of Sale) menggunakan konsep Abstract Class dan Interface pada Java. Program ini digunakan untuk menghitung total pembayaran dari beberapa jenis transaksi, yaitu tiket bioskop reguler, tiket bioskop VIP, dan paket snack.

Program terdiri dari enam class, yaitu ItemTransaksi sebagai interface, TiketBioskop sebagai abstract class, TiketReguler, TiketVIP, PaketSnack, dan Main. Setiap jenis transaksi memiliki perhitungan nominal pembayaran masing-masing, tetapi seluruhnya menerapkan interface ItemTransaksi agar dapat ditampilkan dan dihitung melalui satu array yang sama.

Selain itu, program menerapkan method overriding, default method, dan static method untuk menampilkan rincian transaksi serta mencatat aktivitas kasir. Hasil akhirnya berupa rincian setiap transaksi dan total keseluruhan pembayaran.

Jawaban :

[ItemTransaksi]

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package andita.projectmodul07.unguided;


/**
 *
 * @author andit
 */
public interface ItemTransaksi {


    double getNominalBayar();
```

```
    String getNamaTransaksi();

    // Default method untuk mencetak rincian transaksi
    default void cetakRincianItem() {
        System.out.println("- " + getNamaTransaksi()
                + " | Total: Rp" + getNominalBayar());
    }

    // Static method untuk mencatat aktivitas kasir
    static void catatLogKasir(String pesan) {
        System.out.println("[LOG SISTEM KASIR] " + pesan);
    }
}
```

[TiketBioskop]

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package andita.projectmodul07.unguided;

/**
 *
 * @author andit
 */
public abstract class TiketBioskop implements ItemTransaksi {

    private String judulFilm;
    private double hargaDasar;
    private int jumlahKursi;

    // Constructor
    public TiketBioskop(String judulFilm, double hargaDasar, int jumlahKursi) {
```

```
        this.judulFilm = judulFilm;
        this.hargaDasar = hargaDasar;
        this.jumlahKursi = jumlahKursi;
    }

    // Getter
    public String getJudulFilm() {
        return judulFilm;
    }

    public double getHargaDasar() {
        return hargaDasar;
    }

    public int getJumlahKursi() {
        return jumlahKursi;
    }

    // Abstract method
    public abstract double hitungTotalBayar();

    @Override
    public abstract String getNamaTransaksi();

    // Implementasi interface
    @Override
    public double getNominalBayar() {
        return hitungTotalBayar();
    }
}
```

[TiketReguler]

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package andita.projectmodul07.unguided;

/**
 *
 * @author andit
 */
```

```
public class TiketReguler extends TiketBioskop {

    private int nomorStudio;

    // Constructor
    public TiketReguler(String judulFilm, double hargaDasar,
            int jumlahKursi, int nomorStudio) {

        super(judulFilm, hargaDasar, jumlahKursi);
        this.nomorStudio = nomorStudio;
    }

    // Getter
    public int getNomorStudio() {
        return nomorStudio;
    }

    // Menghitung total pembayaran tiket reguler
    @Override
    public double hitungTotalBayar() {
        return getHargaDasar() * getJumlahKursi();
    }

    // Menampilkan nama transaksi
    @Override
    public String getNamaTransaksi() {
        return "Tiket Reguler (Studio " + nomorStudio + ") - "
                + getJudulFilm() + " (" + getJumlahKursi()
                + " x Rp" + getHargaDasar() + ")";
    }
}
```

[TiketVIP]

```
 /*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package andita.projectmodul07.unguided;

/**
 *
 * @author andit
```

```
 */
public class TiketVIP extends TiketBioskop {

    private double biayaFasilitasVIP;

    // Constructor
    public TiketVIP(String judulFilm, double hargaDasar,
            int jumlahKursi, double biayaFasilitasVIP) {

        super(judulFilm, hargaDasar, jumlahKursi);
        this.biayaFasilitasVIP = biayaFasilitasVIP;
    }

    // Getter
    public double getBiayaFasilitasVIP() {
        return biayaFasilitasVIP;
    }

    // Menghitung total pembayaran tiket VIP
    @Override
    public double hitungTotalBayar() {
        return (getHargaDasar() + biayaFasilitasVIP)
                * getJumlahKursi();
    }

    // Menampilkan nama transaksi
    @Override
    public String getNamaTransaksi() {
        return "Tiket VIP Premiere - " + getJudulFilm()
                + " (" + getJumlahKursi() + " x Rp"
                + (getHargaDasar() + biayaFasilitasVIP) + ")";
    }
}
```

[PaketSnack]

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package andita.projectmodul07.unguided;

/**
```

```
 *
 * @author andit
 */
public class PaketSnack implements ItemTransaksi {

    private String namaPaket;
    private int jumlahPorsi;
    private double hargaPerPorsi;

    // Constructor
    public PaketSnack(String namaPaket, int jumlahPorsi,
            double hargaPerPorsi) {

        this.namaPaket = namaPaket;
        this.jumlahPorsi = jumlahPorsi;
        this.hargaPerPorsi = hargaPerPorsi;
    }

    // Getter
    public String getNamaPaket() {
        return namaPaket;
    }

    public int getJumlahPorsi() {
        return jumlahPorsi;
    }

    public double getHargaPerPorsi() {
        return hargaPerPorsi;
    }

    // Menghitung total pembayaran snack
    @Override
    public double getNominalBayar() {
        return jumlahPorsi * hargaPerPorsi;
    }

    // Menampilkan nama transaksi
    @Override
    public String getNamaTransaksi() {
        return "F&B Snack: " + namaPaket + " ("
                + jumlahPorsi + " x Rp" + hargaPerPorsi + ")";
    }
}
```

[Main]

```
 /*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package andita.projectmodul07.unguided;

/**
 *
 * @author andit
 */
public class Main {

    public static void main(String[] args) {

        // Pengujian proteksi abstract class
        // TiketBioskop tiket = new TiketBioskop("Interstellar", 40000, 2);

        // Membuka kasir
        ItemTransaksi.catatLogKasir(
                "Membuka transaksi baru untuk Pelanggan #042.");

        System.out.println("=== DAFTAR TRANSAKSI BIOSKOP ===");

        // Membuat objek transaksi
        TiketReguler reguler = new TiketReguler(
                "Interstellar", 40000, 2, 3);

        TiketVIP vip = new TiketVIP(
                "Dune: Part Two", 60000, 2, 25000);

        PaketSnack snack = new PaketSnack(
                "Combo Popcorn Caramel + 2 Soda", 1, 55000);

        // Array polimorfik
        ItemTransaksi[] items = new ItemTransaksi[3];

        items[0] = reguler;
        items[1] = vip;
        items[2] = snack;

        // Menghitung total seluruh transaksi
        double grandTotal = 0;
```

```
        for (ItemTransaksi item : items) {

            item.cetakRincianItem();

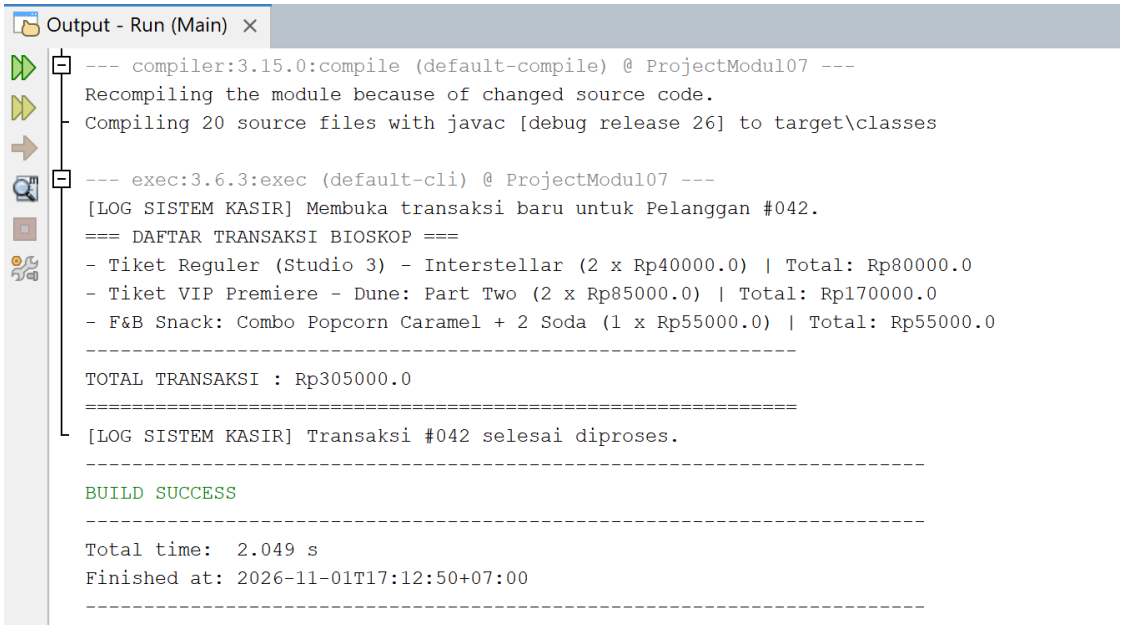
            grandTotal += item.getNominalBayar();
        }

        // Menampilkan grand total
        System.out.println("------------------------------------------------------------");
        System.out.println("TOTAL TRANSAKSI : Rp" + grandTotal);

System.out.println("============================================================
==================");

        // Menutup kasir
        ItemTransaksi.catatLogKasir(
                "Transaksi #042 selesai diproses.");
    }
}
```

Output :



Penjelasan :

program Cinema POS (Point of Sale) dibuat dengan menerapkan konsep Abstract Class dan Interface pada Java. Interface ItemTransaksi digunakan sebagai dasar untuk mengatur method yang harus dimiliki setiap jenis transaksi, yaitu getNominalBayar() dan getNamaTransaksi(). Sementara itu, abstract class TiketBioskop berfungsi sebagai kelas induk yang menyimpan atribut umum tiket bioskop, seperti judul film, harga dasar, dan jumlah kursi. Class TiketReguler dan TiketVIP merupakan turunan dari TiketBioskop yang memiliki perhitungan total pembayaran berbeda sesuai jenis tiketnya.

Selain itu, class PaketSnack dibuat untuk merepresentasikan transaksi pembelian makanan dan minuman yang juga mengimplementasikan interface ItemTransaksi. Pada class Main, dibuat objek dari masing-masing jenis transaksi, kemudian seluruh objek disimpan ke dalam array bertipe ItemTransaksi[]. Program selanjutnya menampilkan rincian setiap transaksi menggunakan default method cetakRincianItem(), menghitung total seluruh pembayaran, serta menggunakan static method catatLogKasir() untuk mencatat aktivitas sistem kasir. Dengan demikian, program dapat menampilkan rincian transaksi beserta total pembayaran keseluruhan.

D. Kesimpulan

Berdasarkan praktikum Modul 7 tentang Abstract Class dan Interface, dapat disimpulkan bahwa kedua konsep tersebut digunakan untuk mengatur struktur dan perilaku class dalam pemrograman Java. Abstract class dapat digunakan sebagai kelas induk yang memiliki atribut dan method abstrak untuk diimplementasikan oleh subclass. Sementara itu, interface berfungsi sebagai aturan yang harus diterapkan oleh class yang mengimplementasikannya. Melalui latihan guided, konsep tersebut dipelajari melalui penggunaan abstract class, interface, method overriding, default method, dan static method.

Pada tugas unguided, konsep tersebut diterapkan dalam pembuatan program Cinema POS yang terdiri dari transaksi tiket reguler, tiket VIP, dan paket snack. Program menggunakan abstract class TiketBioskop sebagai kelas induk serta interface ItemTransaksi sebagai dasar pengelolaan berbagai jenis transaksi. Seluruh objek transaksi dapat disimpan dalam satu array bertipe ItemTransaksi[], kemudian ditampilkan dan dihitung total pembayarannya. Dengan praktikum ini, mahasiswa dapat memahami penerapan abstract class dan interface serta penggunaannya dalam membangun program Java yang lebih terstruktur.